22.3 工具权限与策略

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~900k tokens，输出 ~85k tokens（本章合计）

OpenClaw 的工具权限系统是安全架构的关键部分。不同用户、不同场景、不同 Agent 需要的工具集可能差异很大。本节详解 `ToolPolicy` 配置体系及其层级过滤机制。

22.3.1 `ToolPolicy` 配置体系

策略的基本结构

工具策略由 `allow`（允许列表）和 `deny`（拒绝列表）两个维度组成：

```
// src/agents/pi-tools.policy.ts
type SandboxToolPolicy = {
  allow?: string[];    // 允许的工具（白名单模式）
  deny?: string[];     // 拒绝的工具（黑名单模式）
};
```

评估规则：

1. 如果工具名在 `deny` 中 → **拒绝**
2. 如果 `allow` 为空 → **允许**（无白名单等于全部允许）
3. 如果工具名在 `allow` 中 → **允许**
4. 否则 → **拒绝**

```
function makeToolPolicyMatcher(policy: SandboxToolPolicy) {
  const deny = compilePatterns(policy.deny);
  const allow = compilePatterns(policy.allow);
  return (name: string) => {
    const normalized = normalizeToolName(name);
    if (matchesAny(normalized, deny)) return false;   // deny 优先
    if (allow.length === 0) return true;               // 无 allow = 全通过
    return matchesAny(normalized, allow);              // 在 allow 中才通过
  };
}
```

工具组（Tool Groups）

为了简化配置，OpenClaw 定义了一组**工具组**，可以在 `allow` / `deny` 中通过组名引用一批工具：

```
// src/agents/tool-policy.ts
export const TOOL_GROUPS: Record<string, string[]> = {
  "group:memory":     ["memory_search", "memory_get"],
  "group:web":        ["web_search", "web_fetch"],
  "group:fs":         ["read", "write", "edit", "apply_patch"],
  "group:runtime":    ["exec", "process"],
  "group:sessions":   ["sessions_list", "sessions_history", "sessions_send",
                        "sessions_spawn", "session_status"],
  "group:ui":         ["browser", "canvas"],
  "group:automation": ["cron", "gateway"],
  "group:messaging":  ["message"],
  "group:nodes":      ["nodes"],
  "group:openclaw":   [/* 所有 OpenClaw 原生工具 */],
};
```

配置示例：

```
# 只允许文件系统和执行工具
tools:
  allow:
    - "group:fs"
    - "group:runtime"
```

工具配置档案（Tool Profiles）

OpenClaw 预定义了四个配置档案，覆盖常见使用场景：

```
const TOOL_PROFILES: Record<ToolProfileId, ToolProfilePolicy> = {
  minimal: {
    allow: ["session_status"],          // 仅状态查看
  },
  coding: {
    allow: ["group:fs", "group:runtime", "group:sessions",
            "group:memory", "image"],   // 开发场景
  },
  messaging: {
    allow: ["group:messaging", "sessions_list", "sessions_history",
            "sessions_send", "session_status"],  // 纯消息场景
  },
  full: {},                             // 无限制
};
```

使用方式：

```
tools:
  profile: coding      # 使用 coding 预设
  alsoAllow:
    - web_search        # 在 coding 基础上额外允许 web_search
```

`alsoAllow` 是个巧妙的设计——它允许在 profile 基础上**追加**工具，而不需要完全覆盖 `allow` 列表。

模式匹配

工具名支持 `*` 通配符，由编译后的正则表达式匹配：

```
function compilePattern(pattern: string): CompiledPattern {
  const normalized = normalizeToolName(pattern);
  if (normalized === "*") return { kind: "all" };            // 匹配所有
  if (!normalized.includes("*")) return { kind: "exact", value: normalized };
  // discord_* → 匹配所有 discord 开头的工具
  const escaped = normalized.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
  return { kind: "regex", value: new RegExp(`^${escaped.replaceAll("\\*", ".*")}$`) };
}
```

这让配置变得很灵活：

```yaml
tools:
  deny:
    - "discord_*"       # 禁止所有 Discord 相关工具
    - "sessions_*"      # 禁止所有会话管理工具
```

22.3.2 按发送者身份控制工具权限

Owner-Only 工具

某些敏感工具只允许"所有者"（Owner）使用：

```typescript
// src/agents/tool-policy.ts
const OWNER_ONLY_TOOL_NAMES = new Set<string>(["whatsapp_login"]);

export function applyOwnerOnlyToolPolicy(tools, senderIsOwner) {
  if (senderIsOwner) return tools;  // Owner 不受限制

  return tools
    .map((tool) => {
      if (!isOwnerOnlyToolName(tool.name)) return tool;
      // 非 Owner：替换 execute 为报错函数
      return {
        ...tool,
        execute: async () => { throw new Error("Tool restricted to owner senders."); },
      };
    })
    // 非 Owner：完全移除 Owner-Only 工具
    .filter((tool) => !isOwnerOnlyToolName(tool.name));
}
```

这里用了**双重保护**：即使工具因某种原因没被过滤掉（例如 LLM 缓存了旧的工具列表），execute 也会拒绝执行。

群组场景的发送者控制

在群组聊天中，不同发送者可能有不同的工具权限。这通过 `resolveChannelGroupToolsPolicy()` 实现：

```
// src/config/group-policy.ts（概念）
export function resolveChannelGroupToolsPolicy(params) {
  const { cfg, channel, groupId, senderId, senderName } = params;

  // 查找群组特定的工具策略
  const groupConfig = cfg.channels?.[channel]?.groups?.[groupId];

  // 检查 bySender 配置
  const bySender = groupConfig?.tools?.bySender;
  if (bySender) {
    // 按发送者 ID 或名称匹配策略
    for (const [pattern, policy] of Object.entries(bySender)) {
      if (matchesSender(pattern, { senderId, senderName })) {
        return policy;
      }
    }
  }

  return groupConfig?.tools;
}
```

配置示例：

```
channels:
  telegram:
    groups:
      "-100123456":
        tools:
          allow: ["group:messaging"]          # 默认只允许消息工具
          bySender:
            "admin_user":
              allow: ["*"]                     # 管理员允许所有工具
            "intern_user":
              deny: ["exec", "process"]        # 实习生禁止执行命令
```

22.3.3 多层策略级联

策略应用顺序

OpenClaw 工具策略系统最精巧的地方在于**多层策略级联**。策略从上到下逐层过滤，每层进一步收窄工具集：

```
全部工具
  │
  ├── 1. Owner-Only 过滤（非 Owner 移除敏感工具）
  │
  ├── 2. Profile 策略（tools.profile: coding/messaging/etc.）
  │
  ├── 3. Provider Profile 策略（tools.byProvider.<provider>.profile）
  │
  ├── 4. 全局策略（tools.allow / tools.deny）
  │
  ├── 5. 全局 Provider 策略（tools.byProvider.<provider>.allow/deny）
  │
  ├── 6. Agent 策略（agents.<id>.tools.allow/deny）
  │
  ├── 7. Agent Provider 策略（agents.<id>.tools.byProvider.<provider>.allow/deny）
  │
  ├── 8. 群组策略（channels.<ch>.groups.<gid>.tools.allow/deny）
  │
  ├── 9. 沙箱策略（sandbox.tools.allow/deny）
  │
  └── 10. 子 Agent 策略（DEFAULT_SUBAGENT_TOOL_DENY）
          │
          ▼
      最终工具集
```

在源码中，这个级联清晰可见：

```
// src/agents/pi-tools.ts（末尾）
const toolsByAuthorization = applyOwnerOnlyToolPolicy(tools, senderIsOwner);
const toolsFiltered = filterToolsByPolicy(toolsByAuthorization, profilePolicyExpanded);
const providerProfileFiltered = filterToolsByPolicy(toolsFiltered, providerProfileExpan
const globalFiltered = filterToolsByPolicy(providerProfileFiltered, globalPolicyExpande
const globalProviderFiltered = filterToolsByPolicy(globalFiltered, globalProviderExpand
const agentFiltered = filterToolsByPolicy(globalProviderFiltered, agentPolicyExpanded);
const agentProviderFiltered = filterToolsByPolicy(agentFiltered, agentProviderExpanded)
const groupFiltered = filterToolsByPolicy(agentProviderFiltered, groupPolicyExpanded);
const sandboxed = filterToolsByPolicy(groupFiltered, sandboxPolicyExpanded);
const subagentFiltered = filterToolsByPolicy(sandboxed, subagentPolicyExpanded);
```

按 Provider 差异化策略

不同模型提供者可能有不同的工具需求。例如，某些工具在使用 OpenAI 模型时不可用：

```yaml
tools:
  byProvider:
    openai:
      deny: ["browser"]           # OpenAI 模型禁用浏览器工具
      profile: coding             # 限制为编码工具集
    anthropic:
      profile: full               # Anthropic 模型使用完整工具集
    "openai/gpt-4":               # 可以精确到模型级别
      allow: ["group:fs", "exec"]
```

策略解析时会按 `provider/model` → `provider` 的顺序查找最具体的匹配：

```
function resolveProviderToolPolicy(params) {
  const candidates = [
    fullModelId,        // "openai/gpt-4"（最具体）
    normalizedProvider,  // "openai"（通用）
  ];
  for (const key of candidates) {
    const match = lookup.get(key);
    if (match) return match;
  }
  return undefined;
}
```

插件工具的特殊处理

当 `allow` 列表中**只包含插件工具名**（没有任何核心工具名）时，OpenClaw 会认为这是配置错误——用户可能只是想额外添加插件工具，而不是禁用所有核心工具：

```
// src/agents/tool-policy.ts
export function stripPluginOnlyAllowlist(policy, groups, coreTools) {
  // 检查 allow 中是否有任何核心工具
  let hasCoreEntry = false;
  for (const entry of normalized) {
    const expanded = expandToolGroups([entry]);
    if (expanded.some((tool) => coreTools.has(tool))) {
      hasCoreEntry = true;
    }
  }

  // 如果 allow 中没有核心工具，则忽略此 allow 列表
  if (!hasCoreEntry) {
    logWarn("allowlist contains only plugin tools. Use tools.alsoAllow for additive.");
    return { policy: { ...policy, allow: undefined }, strippedAllowlist: true };
  }
}
```

这个保护机制避免了一个常见陷阱：用户在 `tools.allow` 中只写了 `["memory_search"]` ，结果意外禁用了所有内置工具。正确做法是用 `tools.alsoAllow` ：

```
# 错误：会禁用所有内置工具（被安全保护纠正）
tools:
  allow: ["memory_search"]

# 正确：在默认工具基础上追加
tools:
  alsoAllow: ["memory_search"]
```

本节小结

1. **工具策略由 `allow` （白名单）和 `deny` （黑名单）组成**，deny 优先级高于 allow。支持工具组（ `group:fs` ）和通配符（ `discord_*` ）简化配置。

2. **四个预定义 Profile** 覆盖常见场景： `minimal` （仅状态查看）、 `coding` （开发场景）、 `messaging` （纯消息）、 `full` （无限制）。 `alsoAllow` 支持在 Profile 基础上追加工具。
3. **Owner-Only 机制使用双重保护**（移除工具 + 替换 execute），敏感工具只对所有者可用。群组场景支持按发送者 ID 精确控制工具权限。
4. **10 层策略级联**从 Owner-Only 到子 Agent 策略逐层过滤，支持按 Provider/Model 差异化配置。插件工具有特殊的安全保护，防止意外禁用核心工具。